第 7 小时

软件工程基础知识

7.0　章节考点分析

第 7 小时主要学习软件工程、需求工程、系统分析与设计、净室软件工程、基于构件的软件工程、软件项目管理等内容。

根据考试大纲，本小时知识点会涉及单项选择题和下午案例分析题，约占 8～15 分，论文也会有涉及。本小时内容较基础，侧重于概念知识和管理知识。根据以往全国计算机技术与软件专业技术资格（水平）考试的出题规律，考查的知识点多来源于教材，扩展内容较少。本小时知识架构如图 7.1 所示。

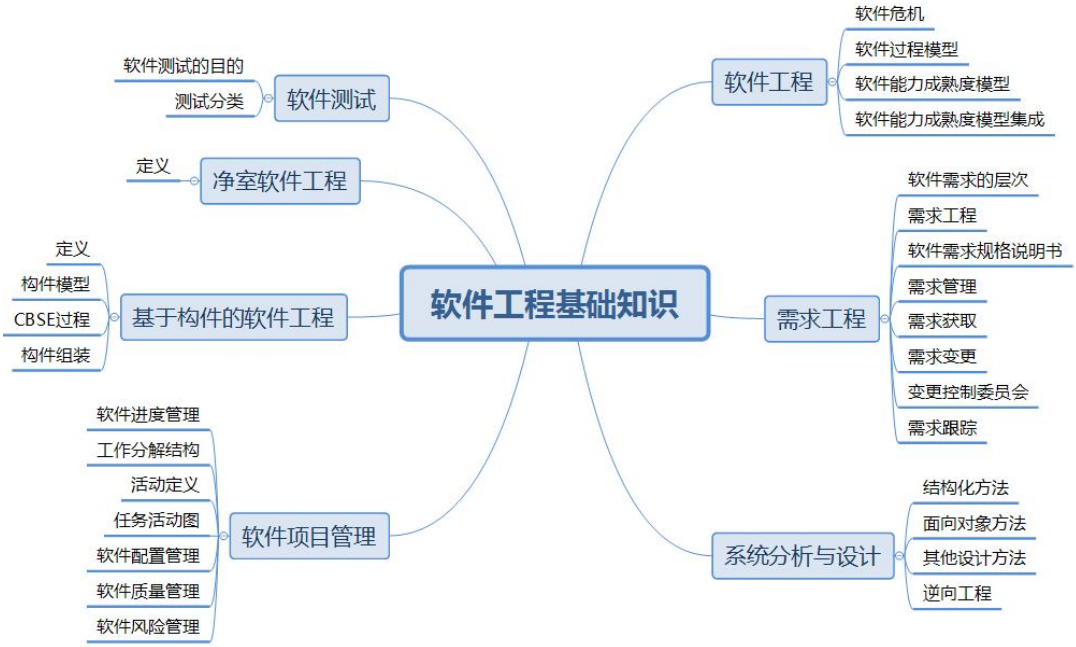

图 7.1　本小时知识架构

【导读小贴士】

软件工程是以工程的管理方法去管理软件项目，涉及的知识点很多，也很重要。像结构化和面向对象在三门考试中都会出题，日常开发中也常常需要面对，所以掌握本小时知识点很重要。本小时的内容比较基础，难度并不大。

7.1 软件工程

【基础知识点】

（1）软件危机（Software Crisis）。具体表现为：软件开发进度难以预测、软件开发成本难以控制、软件功能难以满足用户期望、软件质量无法保证、软件难以维护和软件缺少适当的文档资料。

（2）软件过程模型。软件要经历从需求分析、软件设计、软件开发、运行维护，直至被淘汰这样的全过程，这个全过程就是软件的生命周期。软件生命周期描述了软件从生到死的全过程。为了使软件生命周期中的各项任务能够有序地按照规程进行，需要一定的工作模型对各项任务给予规程约束，这样的工作模型被称为软件过程模型，有时也称为软件生命周期模型。常见的软件过程模型主要包括：

1）瀑布模型（Waterfall Model），如图 7.2 所示，是结构化开发方法使用的软件过程模型。瀑布模型的特点是因果关系紧密相连，前一个阶段工作的输出结果，是后一个阶段工作的输入。每一个阶段工作完成后都伴随着一个里程碑。缺点是需求难以一次确定、变更的代价高、结果难以预见、各阶段工作不能并行。

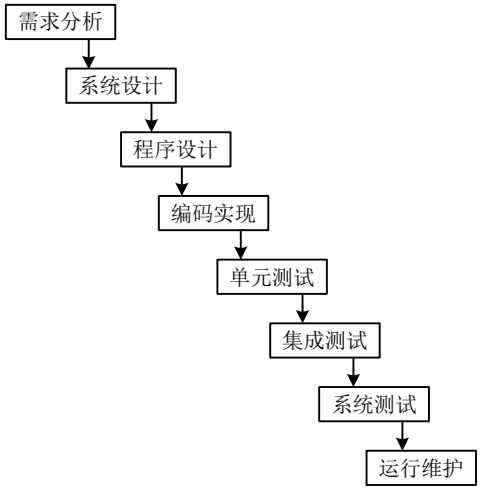

图 7.2　瀑布模型

2）原型模型（Prototype Model），如图 7.3 所示，又称快速原型，是原型方法使用的生命周期模型。原型模型解决了瀑布模型需求难以一次确定、结果难以预见的缺点。原型模型有原型开发和目标软件开发两个阶段。抛弃型原型将原型作为需求确认的手段，在需求确认结束后就被抛弃不用，继续用瀑布模型。演化性原型在需求确认结束后，不断补充和完善原型，直至形成一个完整的产品。

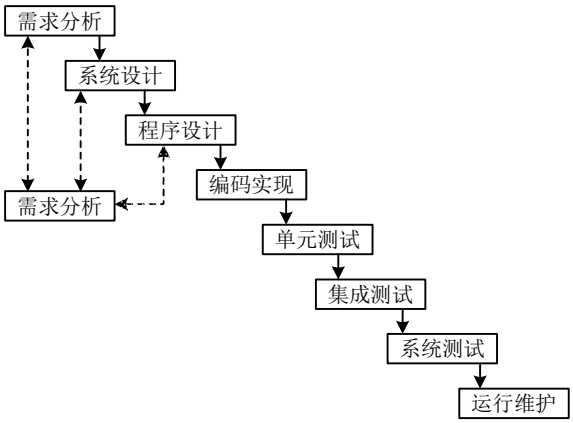

图 7.3　原型模型

3）螺旋模型（Spiral Model），如图 7.4 所示，是在快速原型的基础上结合瀑布模型扩展而成。把整个软件开发流程分成多个阶段，每一个阶段都由目标设定、风险分析、开发和有效性验证、评审 4 部分组成。支持大型软件开发，适用于面向规格说明、面向过程和面向对象的软件开发方法，强调其他模型忽视的风险分析。

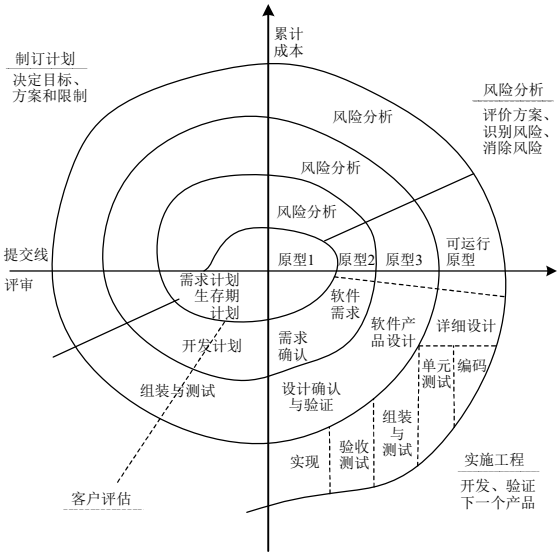

图 7.4　螺旋模型

4）敏捷（Agile）模型，属于敏捷方法使用的模型。敏捷模型主要有极限编程（Extreme Programming，XP）、水晶系列方法、并列争球法（Scrum）、特征驱动开发方法（Feature Driven Development，FDD）等具体的敏捷方法，这些方法的显著特征如下：

- 极限编程（XP）：高效、低风险、测试先行（先写测试代码，再编写程序）。
- 水晶系列方法：不同的项目，采用不同的策略。
- 并列争球法（Scrum）：该方法侧重于项目管理。Scrum 包括一系列实践和预定义角色的过程骨架（是一种流程、计划、模式，用于有效率地开发软件）。在 Scrum 中，使用产品 Backlog 来管理产品的需求，产品 Backlog 是一个按照商业价值排序的需求列表。根据 Backlog 的内容，将整个开发过程分为若干个短的迭代周期（Sprint），在 Sprint 中，Scrum 团队从产品 Backlog 中挑选最高优先级的需求组成 Sprint Backlog。在每个迭代结束时，Scrum 团队将递交潜在可交付的产品增量。当所有 Sprint 结束时，团队提交最终的软件产品。
- 特征驱动开发方法：该方法会将开发人员分类，分为指挥者（首席程序员）、类程序员等。

5）软件统一过程（Rational Unified Process，RUP）模型。RUP 是一种重量级过程模型，属于构件化开发使用的软件过程模型。其生命周期是一个二维的软件开发模型，划分为多个循环（Cycle），每个循环生成产品的一个新的版本，每个循环依次由初始、细化、构造和移交 4 个连续的阶段（Phase）组成，每个阶段完成确定的任务。RUP 中有 9 个核心工作流，这 9 个核心工作流分别是：业务建模、需求、分析与设计、实现、测试、部署、配置与变更管理、项目管理、环境。RUP 的特点是用例驱动的、以架构为中心的、迭代和增量的软件开发过程。

RUP 用“4+1”视图模型来描述架构，如图 7.5 所示，后被 UML 吸收采纳。

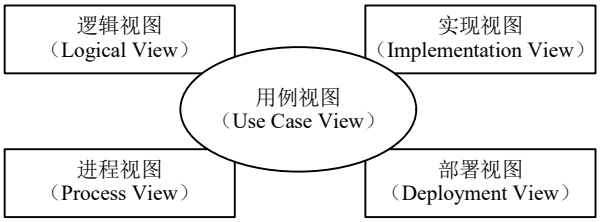

图 7.5　“4+1”视图模型

- 逻辑视图：对应最终用户，主要支持功能性需求，即在为用户提供服务方面系统所应该提供的功能。逻辑视图常用类图、对象图、状态图、协作图表示。
- 实现视图：又称为开发视图，对应程序员，关注软件开发环境下实际模块的组织，描述系统的各部分如何被组织为模块和组件即开发环境中软件的静态组织结构。该视图通常包含包图和组件图。
- 进程视图：又叫过程视图，对应系统集成人员，考虑一些非功能性的需求，如性能和可用性，它可以解决并发性、分布性、系统完整性、容错性的问题。进程视图常用活动图表示。
- 部署视图：又叫物理视图，对应系统工程师。描述如何将前三个视图中所述的系统设计实现为一组现实世界的实体。展示了如何把软件映射到硬件上，它通常要考虑到系统性能、

规模、可靠性等。解决系统拓扑结构、系统安装、通信等问题。部署视图常用部署图表示。

- 用例视图：所有其他视图都依靠用例视图（场景）来指导它们，这就是将模型称为“4+1”的原因。

RUP 在每次迭代中，只考虑系统的一部分需求，进行分析、设计、实现、测试和部署等过程。

（3）软件能力成熟度模型（Capability Maturity Model for Software，CMM）。CMM 是一个概念模型，模型框架和表示是刚性的，不能随意改变，但模型的解释和实现有一定弹性。

（4）软件能力成熟度模型集成（Capability Maturity Model Integration for Software，CMMI）。CMMI 是在 CMM 的基础上发展而来的。CMMI 提供了一个软件能力成熟度的框架，它将软件过程改进的步骤组织成 5 个成熟度等级：初始级、已管理级、已定义级、量化管理级、优化级。量化管理级与已定义级的区别是对过程性能的可预测。

7.2 需求工程

【基础知识点】

1. 软件需求的层次

软件需求包括 3 个不同的层次。

（1）业务需求（Business Requirement），反映了组织机构或客户对系统、产品高层次的目标要求。

（2）用户需求（User Requirement），描述了用户使用产品必须要完成的任务，是用户对该软件产品的期望。业务需求和用户需求构成了用户原始需求文档的内容。

（3）功能需求（functional requirement），从系统操作的角度定义了开发人员必须实现的软件功能，来满足业务需求和用户需求。

2. 需求工程（Requirement Engineering，RE）

需求工程是指应用已证实有效的原理、方法，通过合适的工具和记号，系统地描述待开发系统及其行为特征和相关约束。需求工程由需求获取、需求分析、形成需求规格（或称为需求文档化）、需求确认与验证、需求管理 5 个阶段组成。

3. 软件需求规格说明书（Software Requirement Specification，SRS）

SRS 具体包括功能需求、非功能需求和约束。约束包括设计约束和过程约束。批准的 SRS 是需求开发和需求管理之间的桥梁。

4. 需求管理

需求管理是一个对系统需求变更、了解和控制的过程，包括变更控制、版本控制、需求跟踪等活动。

5. 需求获取

需求获取是获得系统必要的特征，或者是获得用户能接受的、系统必须满足的约束。需求获取的基本步骤：

（1）开发高层的业务模型。

（2）定义项目范围和高层需求。

（3）识别用户角色和用户代表。

（4）获取具体的需求。

（5）确定目标系统的业务工作流。

（6）需求整理与总结。

需求获取的方法包括用户面谈、需求专题讨论会、问卷调查、现场观察、原型化方法和头脑风暴法等。

6. 需求变更

需求变更管理过程如图 7.6 所示。

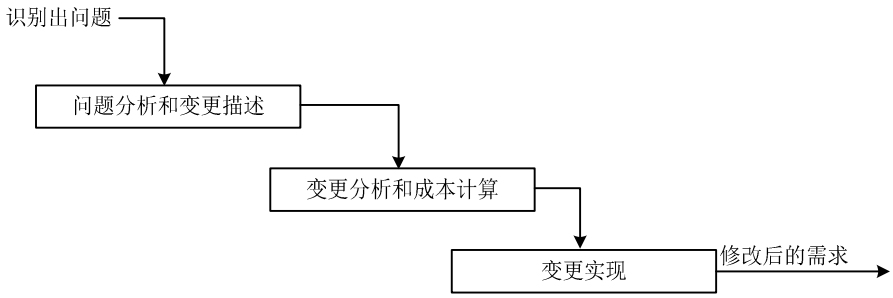

图 7.6　需求变更管理过程

7. 变更控制委员会（Change Control Board，CCB）

CCB 由项目所涉及的多方成员共同组成，通常包括用户和实施方的决策人员。CCB 是决策机构，不是作业机构，通常 CCB 的工作是通过评审手段来决定项目是否能变更，但不提出变更方案。过程及操作步骤为制定决策、交流情况、重新协商约定。

8. 需求跟踪

需求跟踪提供了由需求到产品实现整个过程范围的明确查阅的能力。需求跟踪的目的是建立与维护“需求—设计—编程—测试”之间的一致性，确保所有的工作成果符合用户需求。需求跟踪有正向跟踪和逆向跟踪两种方式，合称为“双向跟踪”。不论采用何种跟踪方式，都要建立与维护需求跟踪矩阵。

7.3　系统分析与设计

【基础知识点】

1. 结构化方法（Structured Analysis and Structured Design，SASD）

结构化方法又称为面向功能的软件开发方法或面向数据流的软件开发方法。针对软件生存周期各个不同的阶段，有结构化分析、结构化设计和结构化编程等方法。

（1）结构化分析（Structured Analysis，SA）。SA 利用图形表达用户需求中的功能需求，使用的手段主要有数据流图（Data Flow Diagram，DFD）、数据字典、结构化语言、判定表以及判定树等。

数据流图（DFD）由 4 种基本元素组成：数据流、处理/加工、数据存储和外部项。

结构化分析具体的建模过程及步骤为明确目标、确定系统范围、建立顶层 DFD 图、构建第一

层 DFD 分解图、开发 DFD 层次结构图、检查确认 DFD 图。DFD 图需要满足规则：父图数据流必须在子图中出现；一个处理至少有一个输入流和一个输出流；一个存储必定有流入和流出；一个数据流至少有一端是处理端；模型表达的信息是全面的、完整的、正确的和一致的。

数据字典（Data Dictionary）是一种标记用户可以访问的数据项和元数据的目录，是对系统中使用的所有数据元素定义的集合，包括数据项、数据结构、数据流、数据存储和处理过程。

（2）结构化设计（Structured Design，SD）。SD 是一种面向数据流的设计方法，以 SRS 和 SA 阶段所产生的数据流图和数据字典等文档为基础，是一个自顶向下、逐步求精和模块化的过程。SD 分为概要设计和详细设计两个阶段，其中概要设计的主要任务是确定软件系统的结构，对系统进行模块划分，确定每个模块的功能、接口和模块之间的调用关系；详细设计的主要任务是为每个模块设计实现的细节。

在 SD 中，模块是实现功能的基本单位，一般具有功能、逻辑和状态 3 个基本属性。

耦合表示模块之间联系的程度，耦合度从低到高依次如表 7.1 所示。

表 7.1　模块的耦合类型

耦合类型	描述
非直接耦合	两个模块之间没有直接关系，互相不依赖对方
数据耦合	一组模块借助参数表传递简单数据
标记耦合	一组模块通过参数表传递记录等复杂信息（数据结构）
控制耦合	模块之间传递的信息中包含用于直接控制模块内部逻辑的信息
通信耦合	一组模块共享了输入或输出
公共耦合	多个模块都访问同一个公共数据环境，公共的数据环境可以是全局数据结构、共享的通信区、内存的公共覆盖区等
内容耦合	一个模块直接访问另一个模块的内部数据、一个模块不通过正常入口跳转到另一个模块的内部、两个模块有一部分程序代码重叠、一个模块有多个入口等

内聚表示模块内部各代码成分之间联系的紧密程度，内聚度从高到低的排序见表 7.2。

表 7.2　模块的内聚类型

内聚类型	描述
功能内聚	各个部分协同完成一个单一功能，缺一不可
顺序内聚	处理元素相关，而且必须顺序执行，通常前一任务的输出是后一任务的输入
通信内聚	所有处理元素集中在一个数据结构的区域上
过程内聚	处理元素相关，而且必须按特定的次序执行
时间内聚	所包含的任务必须在同一时间间隔内执行
逻辑内聚	完成逻辑上相关的一组任务，互相存在调用关系
偶然内聚	完成一组没有关系或松散关系的任务，或者仅仅代码相似

模块分解中应遵循“高内聚、低耦合”的设计原则。

概要设计使用系统结构图（Structure Chart，SC），又称为模块结构图，反映了系统的总体结构。

详细设计的主要任务是设计每个模块的实现算法、所需的局部数据结构。详细设计的表示工具有图形工具、表格工具和语言工具。图形有业务流图、程序流程图、问题分析图（Problem Analysis Diagram，PAD）、NS 流程图等。

（3）结构化编程（Structured Programming，SP）。SP 通过顺序、分支和循环三种基本的控制结构可以构造出任何单入口单出口的程序。SP 强调：自顶向下，逐步细化；清晰第一，效率第二；书写规范，缩进格式；基本结构，组合而成。P 原则：程序=(算法)+(数据结构)。两者分开设计，以算法（函数或过程）为主。

（4）数据库设计（概念结构设计部分）。概念结构设计建立抽象的概念数据模型，通常采用实体-联系图（Entity Relationship Diagram，E-R 图）来表示。

2. *面向对象（Object-Oriented，OO）方法*

面向对象的方法可以分为：

（1）面向对象的分析方法（Object-Oriented Analysis，OOA）。OOA 模型由 5 个层次（主题层、对象类层、结构层、属性层和服务层）和 5 个活动（标识对象类、标识结构、定义主题、定义属性和定义服务）组成。

OOA 的基本原则有抽象、封装、继承、分类、聚合、关联、消息通信、粒度控制和行为分析。OOA 的 5 个基本步骤：确定对象和类、确定结构、确定主题、确定属性、确定方法。

（2）面向对象设计方法（Object-Oriented Design，OOD）。在 OOD 中，数据结构和在数据结构上定义的操作算法封装在一个对象之中。类封装了信息和行为，是具有相同属性、方法和关系的对象集合的总称。类可以分为 3 种类型：

1）实体类：一般来说是一个名词，通常都是永久性需要存储的，例如教师、学生。

2）控制类：是用于控制用例工作的类，控制对象（控制类的实例）通常控制其他对象或协调其他对象的行为，例如登录验证。

3）边界类：用于封装在用例内、外流动的信息或数据流，例如窗口、通信协议、接口等。

（3）面向对象程序设计（Object-Oriented Programming，OOP）。OOP 以对象为核心，该方法认为程序由一系列对象组成。OOP 的基本特点有封装、继承和多态。封装是指将一个计算机系统中的数据以及与这个数据相关的一切操作组装到一起。继承是指一个对象针对于另一个对象的某些独有的特点、能力进行复制或者延续。继承可以分为 4 类，分别为取代继承、包含继承、受限继承和特化继承。多态指同一操作作用于不同的对象，可以产生不同的结果。

（4）数据持久化与数据库。永久保存对象的状态，需要进行对象的持久化（Persistence），把内存中的对象保存到数据库或可永久保存的存储设备中。在多层软件设计和开发中采用持久层（Persistence Layer）专注于实现数据持久化，将对象持久化到关系数据库中，需要进行对象/关系的映射（Object/Relation Mapping，ORM）。目前主流的持久化技术框架包括 Hibernate、iBatis/Mybatis 和 JDO 等。

1）Hibernate：是一个开源的全自动的 ORM 框架，对 JDBC 进行了非常轻量级的对象封装，提供抽象的 HQL 可以自动生成不同数据库的 SQL 语句，优点是具有跨数据库平台的特性。

2）iBatis/Mybatis：提供手动的 ORM 实现，需要程序员手写 SQL，优点是可以结合特定的数据库特性深度优化。

3）Java 数据对象（Java Data Object，JDO）：是 Java 标准中的持久化 API，提供了透明的对象存储，并且不仅仅支持关系数据库，还支持普通文件、XML 文件和对象数据库等。

3. 其他设计方法

其他设计方法如构件与软件重用。软件重用是使用已有软件产品来开发新的软件系统的过程，分为水平式重用和垂直式重用两种类型，见表 7.3。

表 7.3　软件重用类型

名称	对象	举例
水平式重用	不同应用领域中的软件元素	标准函数库
垂直式重用	共性应用领域间的软部件	区块链

4. 逆向工程（Reverse Engineering）

逆向工程是通过分析已有的程序，寻求比源代码更高级的抽象表现形式（比如文档）的活动，是在不同抽象层级中进行的溯源行为。

逆向工程得出的设计称为设计恢复（Design Recovery），但不一定能够抽象还原到原设计。

重构（Restructuring）是在同一抽象层级中转换系统描述的活动。对逆向工程所形成的系统进行修改或重构，生成的新版本称为重构工程。

逆向工程信息恢复的级别见表 7.4。

表 7.4　逆向工程信息恢复的级别

级别	内容	抽象级别	逆向工程恢复难度	工具支持可能性	人工参与程度
实现级	语法树、符号表	递增	递增	递减	递增
结构级	程序分量间的关系，如调用图				
功能级	功能和程序段之间的关系				
领域级	实体与应用域之间的关系				

7.4　软件测试

【基础知识点】

1. 软件测试的目的

测试是确保软件的质量，确认软件以正确的方式做了用户所期望的事情。软件测试通常在规定

的时间和成本内完成，以尽量多地发现漏洞，但不能保证发现所有的漏洞。

2. 测试分类

（1）根据程序执行状态可分为静态测试（Static Testing，ST）和动态测试（Dynamic Testing，DT）。

（2）根据是否关注具体实现和内部结构可分为黑盒测试、白盒测试和灰盒测试。

（3）根据程序执行的方式来分类可分为人工测试（Manual Testing，MT）和自动化测试（Automatic Testing，AT）。

（4）从阶段上划分，软件测试可以分为单元测试、集成测试、系统测试和验收测试。

1）单元测试主要是对该软件的模块进行测试，往往由程序员自己完成。常采用白盒的静态测试如静态分析、代码审查等，也可以采用自动化的动态测试。

2）集成测试对通过单元测试的模块进行组装测试，以验证组装的正确性，一般采用白盒测试和黑盒测试结合的方法。

3）系统测试检查组装完成的系统是否符合 SRS 的要求。主要测试内容包括功能测试、性能测试、健壮性测试、安全性测试等，结束标志是测试工作已满足测试目标所规定的需求覆盖率，并且测试所发现的缺陷都已全部归零。

4）验收测试是确认系统满足用户需求或者协议的要求，确保系统能支撑业务运行。

（5）其他测试还有 AB 测试、Web 测试、链接测试和表单测试等。

7.5 净室软件工程

净室软件工程（Cleanroom Software Engineering，CSE）是一种在软件开发过程中强调在软件中建立正确性的需要的方法。CSE 的理论基础主要是函数理论和抽样理论。CSE 使用盒子结构规约进行分析和设计建模，并且强调将正确性验证（而不是测试）作为发现和消除错误的主要机制，可以生成质量非常高的软件。CSE 的缺点是太理论化、忽视测试、带有传统软件工程的弊端。

7.6 基于构件的软件工程

1. 定义

基于构件的软件工程（Component-Based Software Engineering，CBSE）是一种基于分布对象技术、强调通过可复用构件设计与构造软件系统的软件复用途径。用于 CBSE 的构件应该具备以下特征：

（1）可组装型：所有外部交互必须通过公开定义的接口进行。

（2）可部署性：必须能作为一个独立实体在提供其构件模型实现的构件平台上运行。

（3）文档化：构件必须是完全文档化的。

（4）独立性：构件应该是独立的，如确实需要其他构件提供服务，则应显示声明。

（5）标准化：必须符合某种标准化的构件模型。

2. 构件模型

构件模型定义了构件实现、文档化以及开发的标准。目前主流的构件模型是 Web Services 模型、Sun 公司的 EJB 模型和微软的.NET 模型。构件模型包含了一些模型要素如接口、使用信息和部署信息。构件模型提供了一组被构件使用的通用服务，包括平台服务和支持服务。容器是构件模型基础设施，是支持服务的一个实现加上一个接口定义，构件必须提供该接口定义以便和容器整合在一起。

3. CBSE 过程

支持基于构件组装的软件开发过程主要包括：

（1）系统需求概览。

（2）识别候选构件。

（3）根据发现的构件修改需求。

（4）体系结构设计。

（5）构件定制与适配。

（6）组装构件，创建系统。

CBSE 过程与传统的软件开发过程的不同点：

（1）早期需要完整的需求，以便尽可能多地识别出可复用的构件。

（2）早期阶段根据可利用的构件来细化和修改需求以匹配 CBSE。

（3）架构设计完成后，可能需要修改构件以适合功能和架构的需求。

（4）开发过程就是组装构件的过程，有时需要开发适配器。

（5）CBSE 中的架构设计阶段特别重要，决定和限制了可选构件的范围。

4. 构件组装

常见的构件组装有顺序组装、层次组装和叠加组装 3 种组装方式。构件组装可能面临接口不兼容的问题，常见的有参数不兼容、操作不兼容和操作不完备 3 种。这时需要编写适配器构件来解决不兼容的问题。

7.7 软件项目管理

（1）软件进度管理一般包括活动定义、活动排序、活动资源估计、活动历时估计、制定进度计划和进度控制 6 个过程。

（2）工作分解结构（Work Breakdown Structure，WBS）是把一个项目，按一定的原则分解成任务，任务再分解成一项项工作，再把一项项工作分配到每个人的活动中，直到分解不下去为止。以可交付成果为导向，对项目要素进行的分组，总是处于计划过程的中心。

（3）活动定义是指确定完成项目的各个可交付成果所必须进行的各项具体活动，还需要明确每个活动的前驱、持续时间、必须完成日期、里程碑或可交付成果。

（4）任务活动图是项目进度管理、项目成本管理等一系列项目管理活动的基础，通常采用甘

特图等方式来展示和管理项目活动。

（5）软件配置管理（Software Configuration Management，SCM）是一种标识、组织和控制修改的技术。SCM 的目的是使错误降为最小并最有效地提高生产效率。SCM 的核心内容包括版本控制和变更控制。版本控制（Version Control）是指对软件开发过程中各种文件变更的管理，最主要的功能就是追踪和记录文件的变更、并行开发。变更控制（Change Control）是指对变更进行管理，确保变更有序进行。

（6）软件质量管理。软件质量就是软件与明确地和隐含地定义的需求相一致的程度。软件质量保证（Software Quality Assurance，SQA）的目的是使软件过程对于管理人员来说是可见的。

SQA 的主要任务是：

1）SQA 审计与评审，包括对软件工作产品、软件工具和设备的审计，评审开发组的行为符合预定的过程。

2）SQA 报告。

3）处理不符合问题。

软件质量认证，国内软件企业主要采用的是 ISO 9001 和 CMM。

（7）软件风险管理。软件项目风险是指在软件开发过程中遇到的预算和进度等方面的问题以及这些问题对软件项目的影响。风险管理的主要目标是预防风险，及应对发生的风险。风险管理活动可以分为：

1）Bochm 把风险管理活动分成风险估计（风险辨识、风险分析、风险排序）和风险控制（风险管理计划、风险处理、风险监督）两大阶段。

2）Charette 把风险分成分析（辨识、估计、评价）和管理（计划、控制、监督）两大阶段。

7.8 练习题

1．螺旋模型在（　　）的基础上扩展而成。

A．瀑布模型　　B．原型模型　　C．快速模型　　D．面向对象模型

解析：螺旋模型是在原型模型的基础上扩展而成的。

答案：B

2．__（1）__适用于程序开发人员在地域上分布很广的开发团队。__（2）__中，编程开发人员分成首席程序员和“类”程序员。

（1）A．水晶系列（Crystal）开发方法　　B．开放式源码（Open Source）开发方法

C．Scrum 开发方法　　D．功用驱动开发方法（FDD）

（2）A．自适应软件开发（ASD）　　B．极限编程（XP）开发方法

C．开放系统—过程开发方法（OpenUP）　　D．功用驱动开发方法（FDD）

解析：

（1）在所有的敏捷型方法中，极限编程（Extreme Programming，XP）是最引人瞩目的。它源

于 Smalltalk 圈子，特别是 Kent Beck 和 Ward Cunningham 在 20 世纪 80 年代末的密切合作。XP 在一些对费用控制严格的公司中的使用，已经被证明是非常有效的。

（2）Cockburn 的水晶系列方法。水晶系列方法是由 Alistair Cockburn 提出的，它与 XP 方法一样，都有以人为中心的理念，但在实践上有所不同。Alistair 考虑到人们一般很难严格遵循一个纪律约束很强的过程，因此，与 XP 的高度纪律性不同，Alistair 探索了用最少纪律约束而仍能成功的方法，从而在产出效率与易于运作上达到一种平衡。也就是说，虽然水晶系列不如 XP 产出效率高，但会有更多的人能够接受并遵循它。

（3）开放式源码指的是开放源码界所用的一种运作方式。开放式源码项目有一个特别之处，就是程序开发人员在地域上分布很广，这使得它和其他敏捷方法不同，因为一般的敏捷方法都强调项目组成员在同一地点工作。开放源码的一个突出特点就是查错排障（Debug）的高度并行性，任何人发现了错误都可将改正源码的“补丁”文件发给维护者。然后由维护者将这些“补丁”或是新增的代码并入源码库。

（4）Scrum。Scrum 已经出现很久了，像前面所论及的方法一样，该方法强调这样一个事实，即明确定义了的可重复的方法过程只限于在明确定义了的可重复的环境中，为明确定义了的可重复的人员所用，去解决明确定义了的可重复的问题。

（5）功用驱动开发方法（Feature Driven Development，FDD）是由 Jeff De Luca 和 Peter Coad 提出来的。像其他方法一样，它致力于短时的迭代阶段和可见可用的功能。在 FDD 中，一个迭代周期一般是两周。

在 FDD 中，编程开发人员分成两类：首席程序员和“类”程序员（Class Owner）。首席程序员是最富有经验的开发人员，他们是项目的协调者、设计者和指导者，而“类”程序员则主要做源码编写。

（6）ASD 方法。ASD（Adaptive Software Development）方法由 Jim Highsmith 提出，其核心是三个非线性的、重叠的开发阶段：猜测、合作与学习。

答案：B　D

3．需求管理是一个对系统需求变更、了解和控制的过程。以下活动中，（　　）不属于需求管理的主要活动。

A．文档管理　　B．需求跟踪　　C．版本控制　　D．变更控制

解析：需求管理的活动包括变更控制、版本控制和需求跟踪等。

答案：A

4．结构化程序设计采用自顶向下、逐步求精及模块化的程序设计方法，通过（　　）三种基本的控制结构可以构造出任何单入口单出口的程序。

A．顺序、选择和嵌套　　B．顺序、分支和循环

C．分支、并发和循环　　D．跳转、选择和并发

解析：结构化程序设计采用自顶向下、逐步求精及模块化的程序设计方法，通过顺序、分支和循环三种基本的控制结构可以构造出任何单入口单出口的程序。

答案：B